Monceaux et files de priorité

11 novembre 2024 15:26

TDA : file de priorité

	Arbre balancé	Monceau
• insérer une paire clé-valeur	$O(\log n)$	$O(\log n)$
• obtenir l'elt avec clé minimum	$O(\log n)$	$O(1)$
• supprimer l'elt avec clé minimum	$O(\log n)$	$O(\log n)$

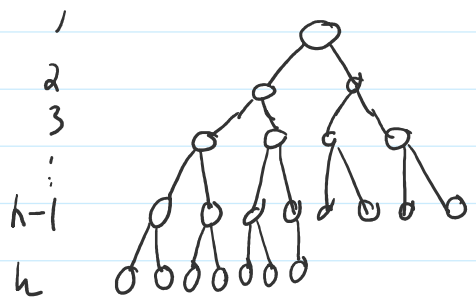
* répétition possible

* répétition possible

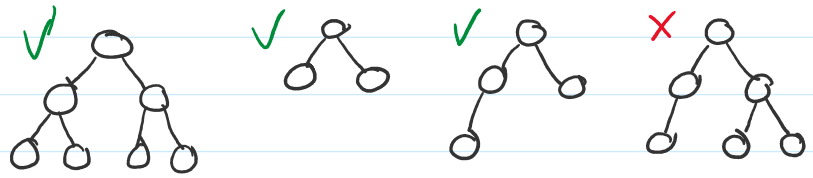
Monceau : arbre binaire où, si la hauteur est h, alors

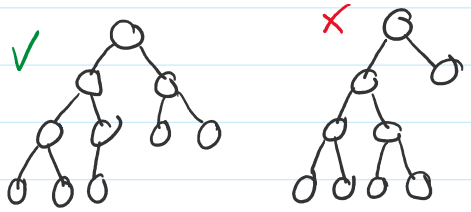
(heap, tas)

- chaque noeud de profondeur $1, 2, \ldots, h-2$ a 2 enfants
- les noeuds de profondeur $h-1$ sont remplis de gauche à droite
  - au plus un noeud avec 1 seul enfant
  - un noeud n'a pas moins d'enfant qu'un autre noeud à sa droite



ex :

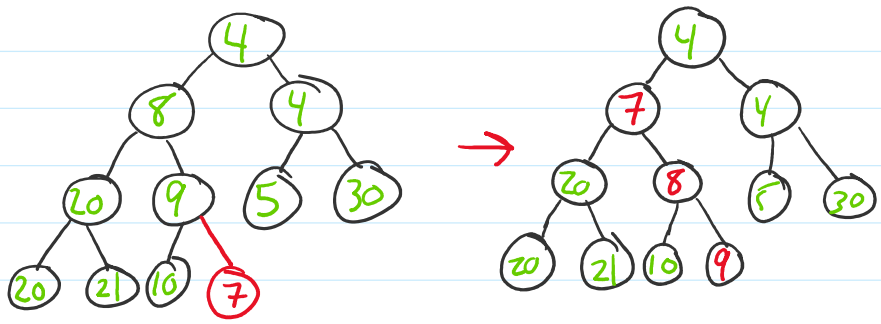




Note : un monceau a une hauteur $O(\log n)$

Monceau-min : pour chaque noeud interne v,

$$\left.\begin{array}{l} v \to clé \leq v \to gauche \to clé \\ \text{et} \\ v \to clé \leq v \to droit \to clé \end{array}\right\} \text{si enfants existent}$$



- get_min trivial : on regarde la racine.

```
std::pair< CLE,VAL> get_min(){
        return std::make_pair(racine->cle, racine->val);
}
```

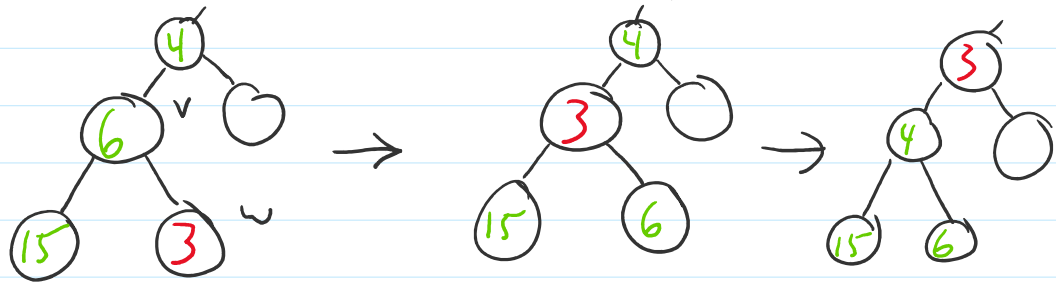
- inserer(cle, val)

  1) trouver v, le prochain noeud à qui il manque un enfant

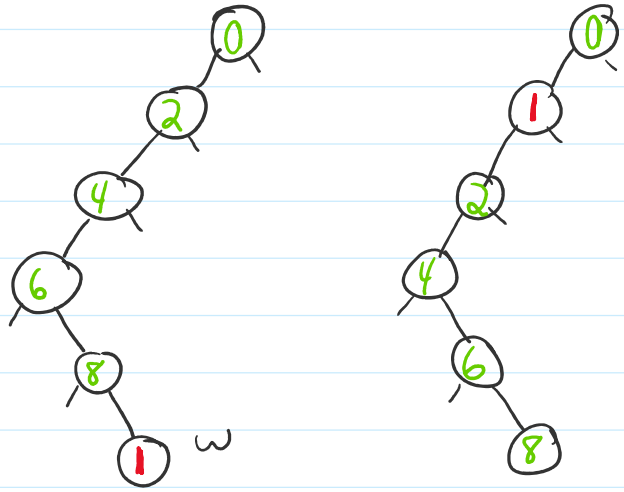
  2) w = new Noeud()
     w->cle = cle
     w->val = val

```
w->val = val
v->ajouter_enfant(w)
```

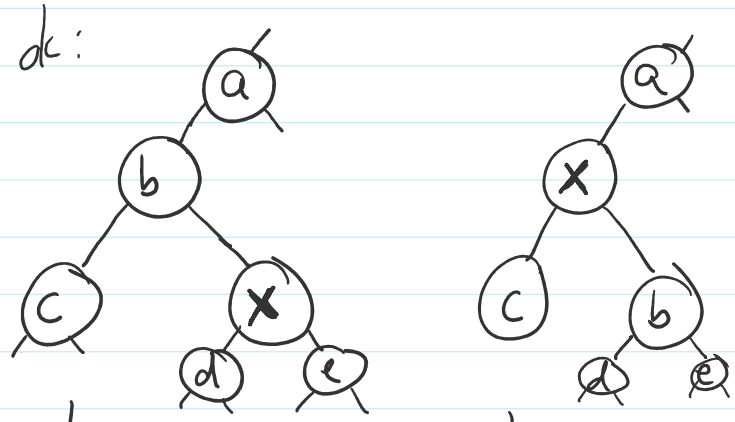
3) faire monter ("percoler") la valeur de w pour avoir les conditions d'un monceau




```
bool done = false;
while (!done){
    if(w == racine)
        done = true
    else if (w->cle < w->parent->cle){
        std::swap(w->cle, w->parent->clé);
        std::swap(w->val, w->parent->val);
        w = w->parent;
    }
    else
        done = true;
}
```



Un swap est ok :



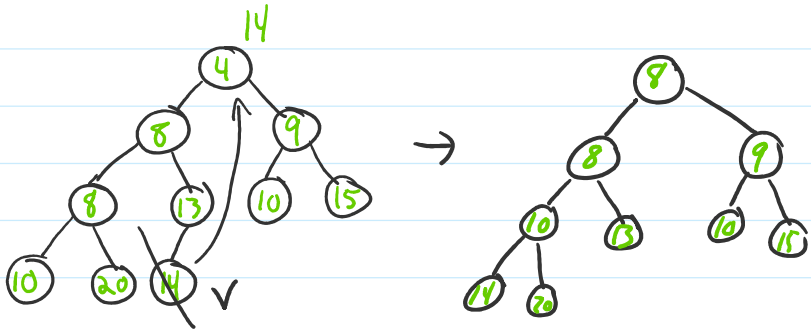
avant d'insérer x :

$x < b$
$c \geq b$
$b \geq a$
$d \geq b$
$e \geq b$

$\rightarrow$

$b > x$ ✓
$c \geq b > x$ ✓
$d \geq b$ ✓
$e \geq b$ ✓

- Supprimer-min

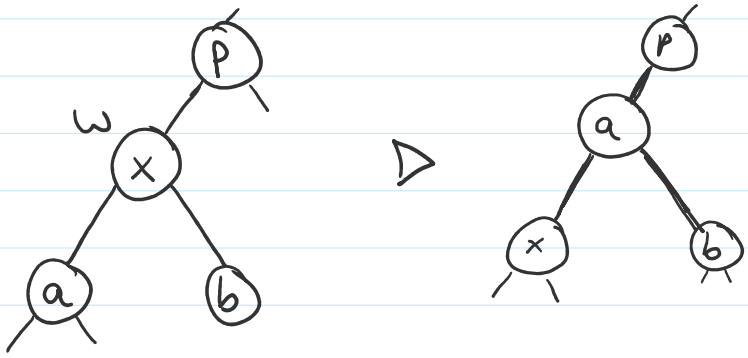


1) Soit v le noeud le plus récemment crée (+ à droite au dernier niveau)

2) Échanger clé/val entre v et racine
3) Delete v
4) Faire descendre l'elt racine ("percolation" ↓) au noeud approprié

```
w = racine;
bool done = false;
while (!done){
```

```
bool done=false;
while(!done){
    if(w->est_feuille())
        done=true;
    else{
        z = l'enfant de w avec la valeur min.
        if(z->cle<w->cle){
            swap w->cle et val avec z->cle et val
            w=z
        }
        else done=true
    }
```



$x \leq p$
$a \leq b$ (disons)
$a < x$

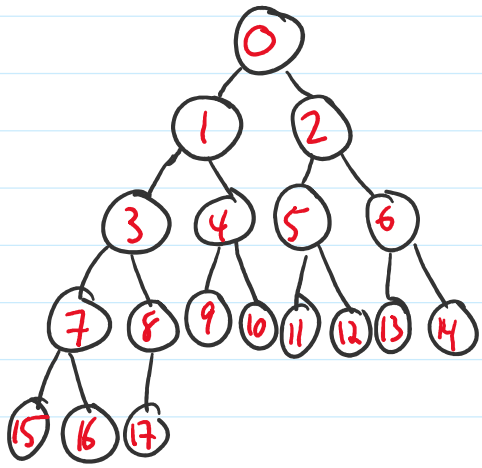
$a < x \leq p$ ✓
$x < a$ ✓
$a \leq b$ ✓

get_min: $O(1)$
insertion: ajouter noeud $O(1)$
percoler ↑ $O(h) = O(\log n)$
supprimer_min: swap racine $O(1)$
percoler ↓ $O(h) = O(\log n)$

Voir aussi: binomial heap
Fibonacci heap

Représentation en tableau

- On numérote les noeuds par niveau de gauche à droite



Enfants du noeud i : $2i+1$ $\quad 2i+2$

- Un tableau suffit : tab[i] = val du noeud i

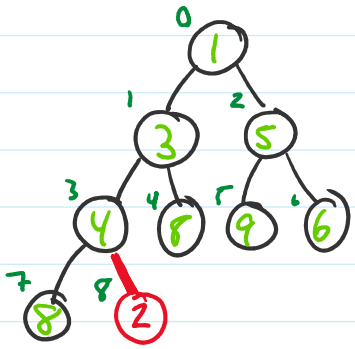
```
get_gauche(i) : tab[2i+1]
get_droit (i) : tab[2i+2]
get_parent(i) : tab[(i-1)/2]      i>0 (plancher)
```

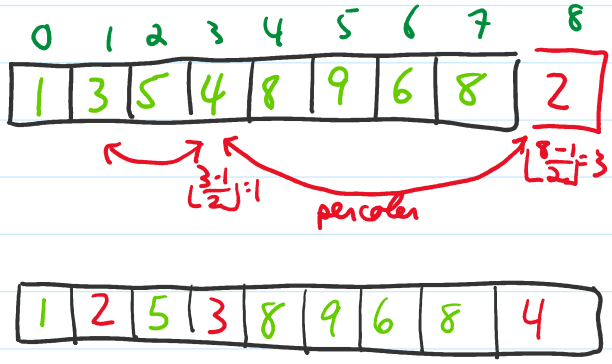
```
class Monceau{

    vector< pair< TCLE, TVAL >>  vec;

    get_min(){ return vec[0]; }

    inserer(cle,val){ vec.push_back( make_pair(cle,val) );
                      percolo dans vec
    }
```





Beaucoup + efficace : localité de mémoire
++ cache

- Heapsort (tab_a_trier)

```
    ret = vector(tab_a_trier.size())
    M = Monceau()

    for(i=0 à tab_a_trier.size())
        M.inserer(tab_a_trier[i]

    for(i=0 à tab_a_trier.size()){
        ret[i] = M.get_min()
        ret.supprimer_min()
    }
```

n fois $O(\log n)$

n fois $O(1) + O(\log n)$

Tri en temps $O(n \log n)$

* Note : pour un ABR ou Monceau,

operator < (plus petit)

doit être défini.

On peut indexer avec des ptr.

On peut indexer avec des ptr.

ex: map<Noeud*, int> mes_noeuds;

set<int*> ints;

Comparera les adresses de ptr.